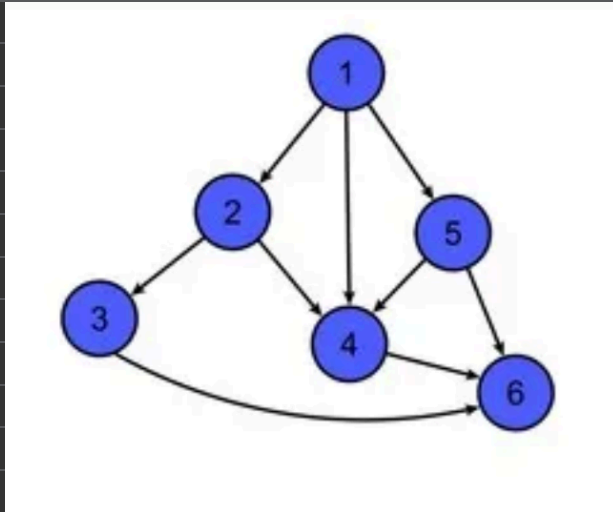


Graph Algorithms

- Directed Acyclic Graphs:

A directed graph with no directed cycle.



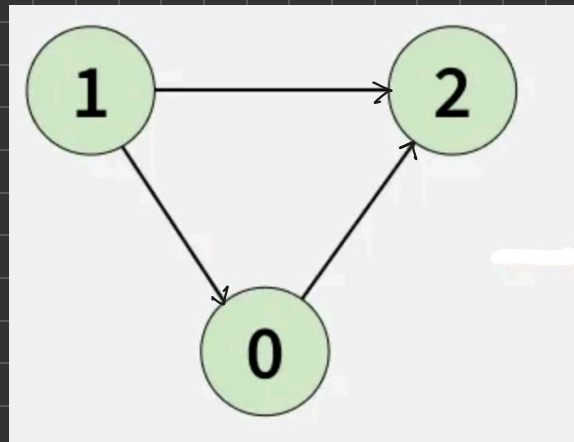
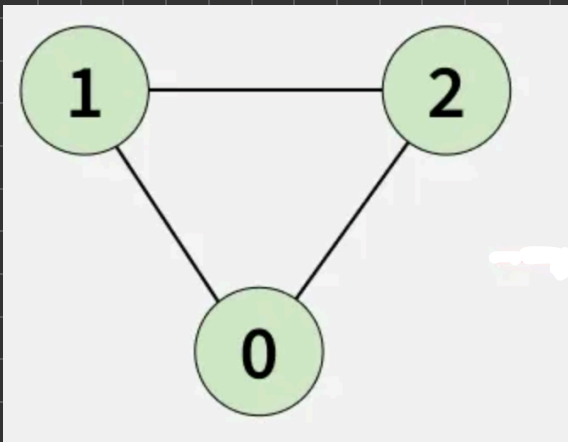
- Undirected Acyclic Graph:

- An undirected graph with no cycles.

- A connected undirected Acyclic graph is a tree & vice versa → a path exists b/w any two nodes

- Cyclic graph:

- A graph with at least one cycle



- Weighted Graph

- Edges have weights

- It is commonly used in optimization problems like shortest path.

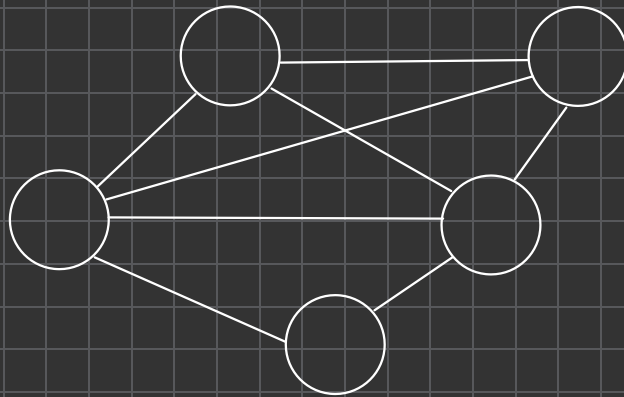
- Unweighted Graph:

- Edges don't have weights

- It is commonly used to represent relationships b/w nodes

- Complete ~~graph~~ graph $\rightarrow$ Dense graph

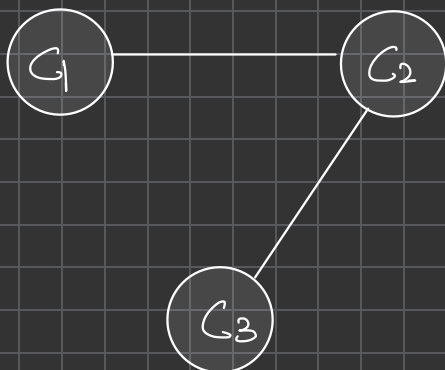
- Every node is connected to every other node



$$O(V \times (V-1))$$

Graph Representation

- Adjacency Matrix



	G_1	G_2	G_3
G_1	0	1	0
G_2	1	0	1
G_3	0	1	0

Bfs

- It is an algorithm for graph/tree traversal.
- It uses a queue data structure
- In BFS, we traverse all nodes level by level & the traversal is completed when all nodes are visited.

Example,

A

B C

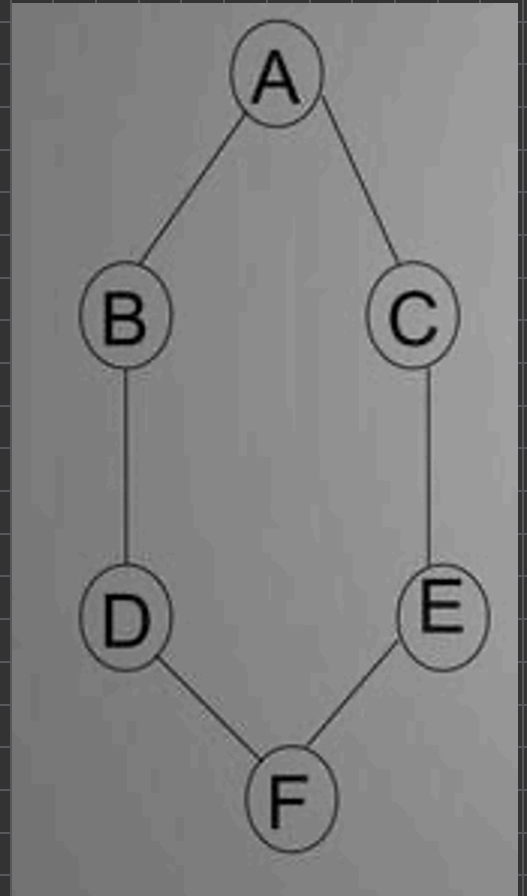
C D

D E

E F

F

A B C D E F



Dfs:

- It is also an algorithm for traversing graph data structures.
- It uses a stack for implementation
- Dfs starts at root & explores as far as possible along each branch before backtracking

Example

A B D F E C
C C C C C

A B D F E C

DFS

- DFS is more faster then BFS.
- DFS requires less memory compare to BFS.
- DFS is not so useful in finding shortest path.
- Example :

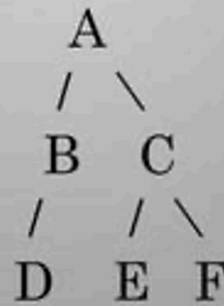


Ans : A,B,D,C,E,F

This slide is true but it is more tree

BFS

- BFS is slower than DFS.
- BFS requires more memory compare to DFS.
- BFS is useful in finding shortest path.
- Example :



Ans : A,B,C,D,E,F

Time Complexity:

$O(V+E)$: Adjacency List

$O(V^2)$: Adjacency Matrix

Space Complexity

$O(V)$ → For Dfs it's longest path

→ For Bfs it's maximum degree

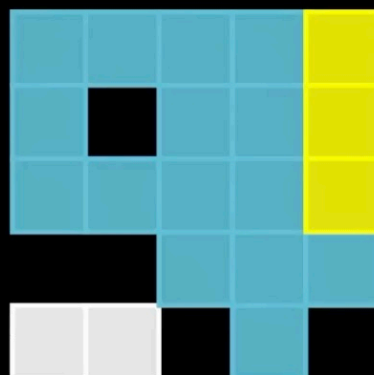
- Tracking node state (unvisited, Processing, Processed) is important for cycle detection & other advanced graph algos.

Applications

Flood Fill Problem

Given an image represented by a grid of pixel values, a starting pixel (row, col), and new pixel value p , transform all pixels **connected** to the starting pixel to the new pixel value.

3	3	3	3	5
3	0	3	3	5
3	3	3	3	5
0	0	3	3	3
1	1	0	3	0



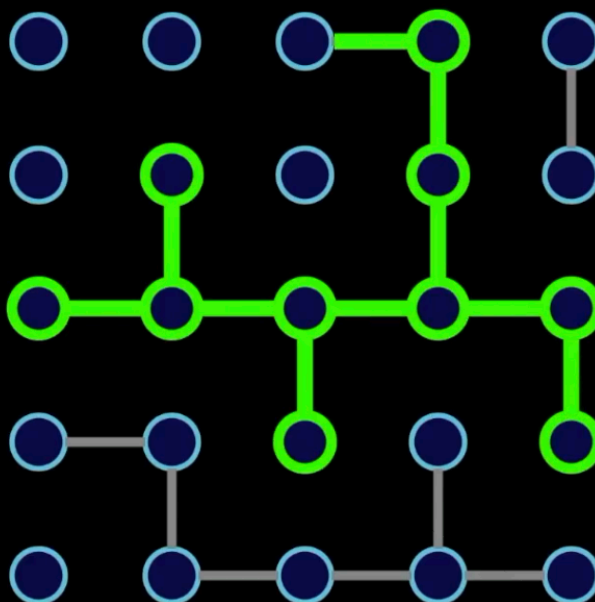
$(\text{row}, \text{col}): (2, 2) \quad p : 3$
 $(\text{row}, \text{col}): (2, 4) \quad p : 5$



Connected pixels share the same value and have a path between them by moving either left, right, up, and down.

1	0	1	2	0
0	2	0	2	0
2	2	2	2	2
0	0	2	0	2
1	0	0	0	0

$(\text{row}, \text{col}): (2, 2) \quad p : 2$



Both Bfs & Dfs can be used to solve this

Dfs Applications

```

procedure Iterative_DFS_Cycle_Detect(G, s)
    // Initialize all vertices
    for each vertex u in G
        u.status = notVisited / white

    let S be an empty Stack

    // Stack will store pairs: (current_vertex, parent_vertex)
    S.push((s, NULL))

    while S is not empty do
        (u, parent) = S.pop()

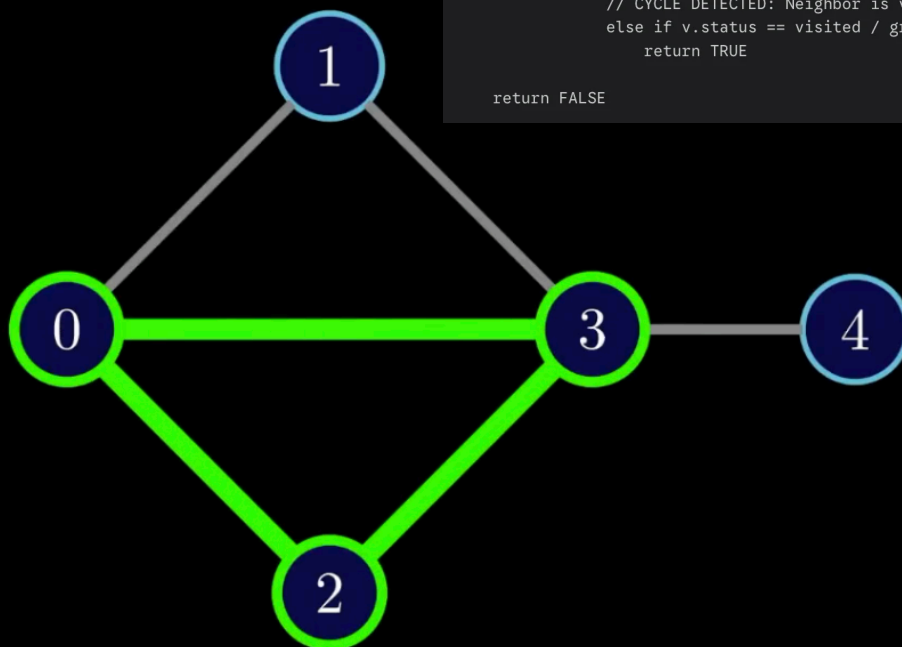
        if u.status == notVisited / white
            for each v in G[u]
                S.push((v, u))
    
```

Cycle

Detection

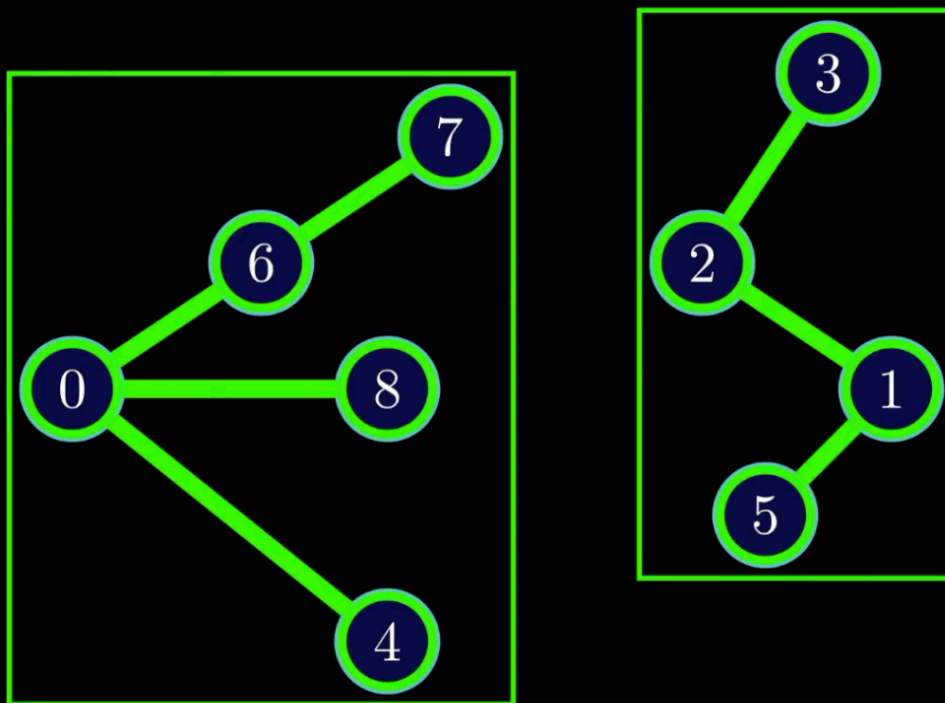
Cycle D

```
u.status = visited / grey  
  
for each v adjacent to u do  
    if v.status == notVisited / white  
        // Push neighbor and set 'u' as its parent  
        S.push((v, u))  
  
    // CYCLE DETECTED: Neighbor is visited AND is not the parent  
    else if v.status == visited / grey AND v != parent  
        return TRUE  
  
return FALSE
```



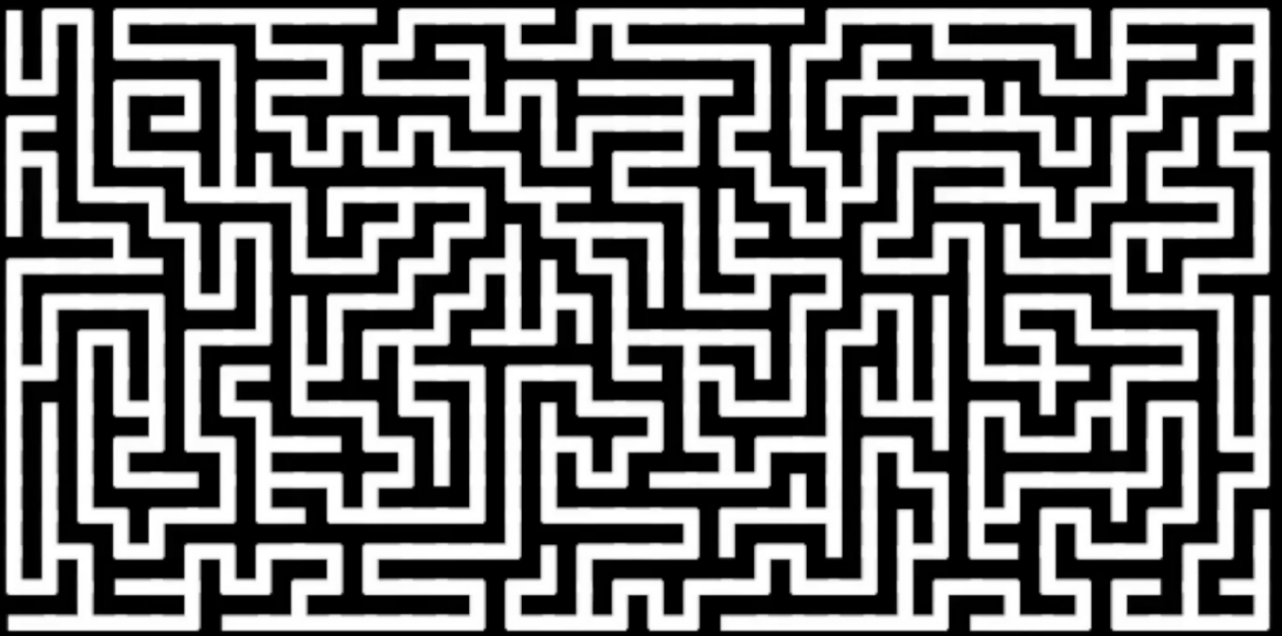
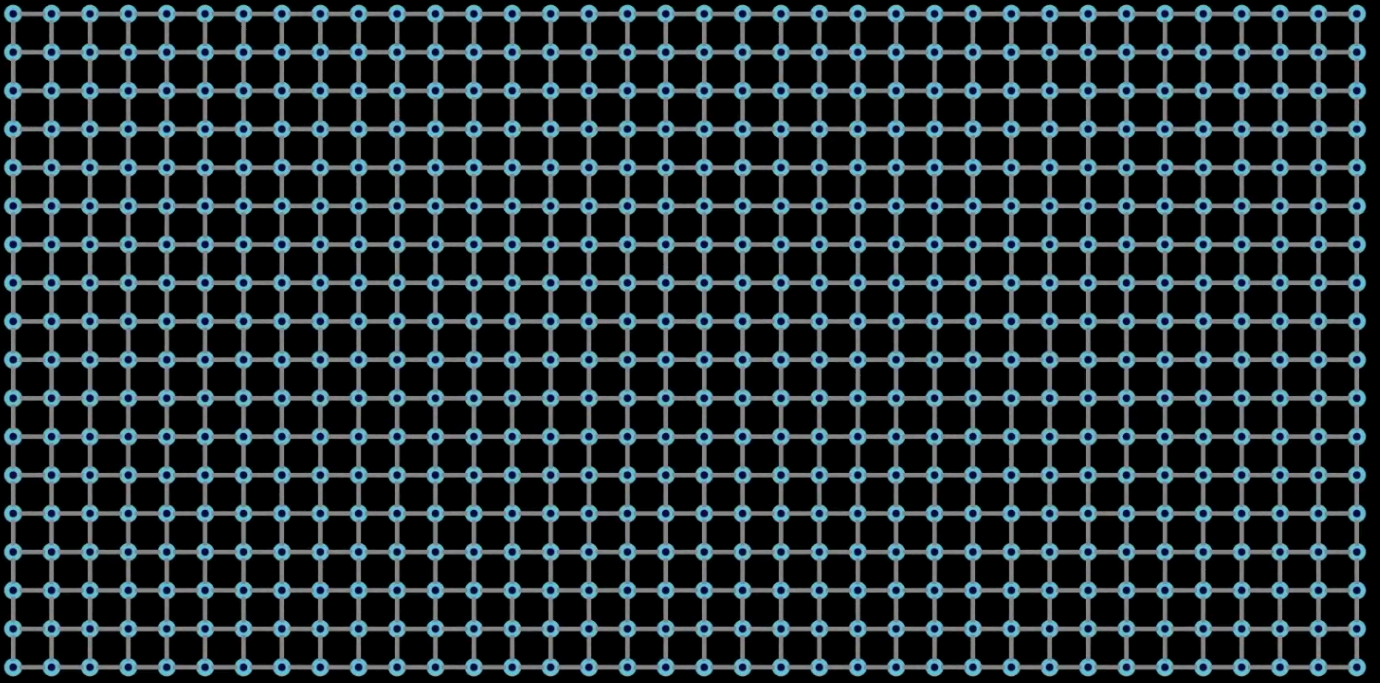
- If a new edge points to a already visited vertex, you have a cycle.

Finding Connected Components



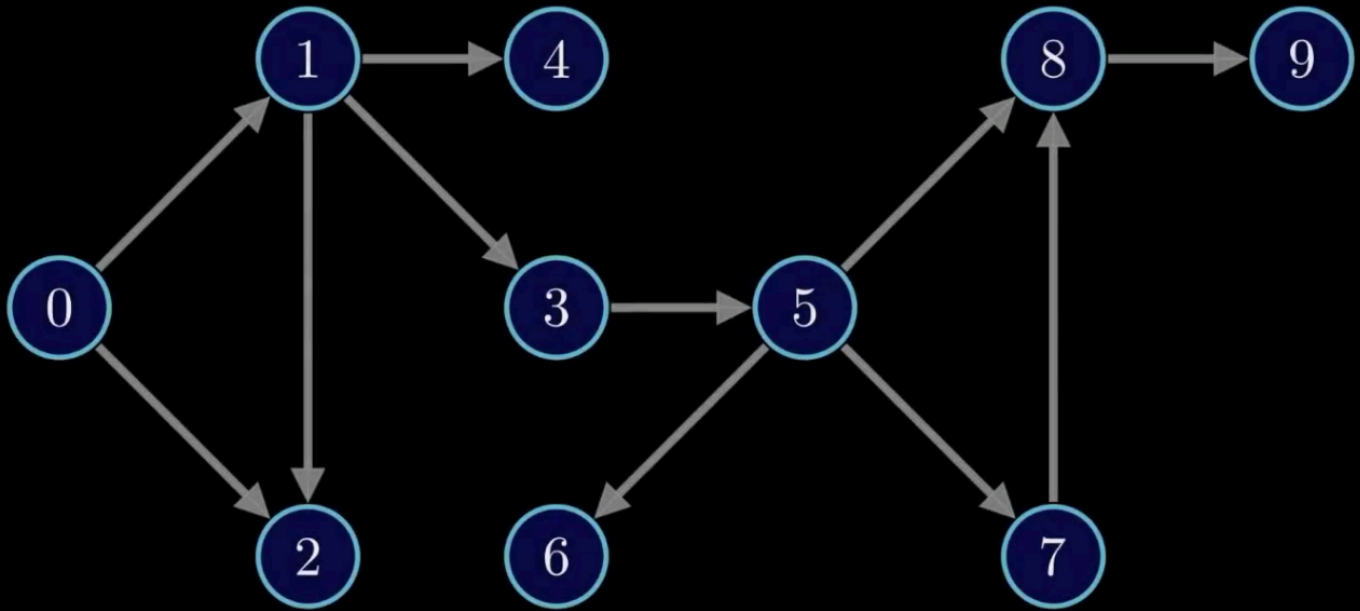
- Run Dfs on a Random vertex, once no more nodes are reachable, it forms one component
- Run Dfs on an unvisited vertex
- Repeat till no unvisited vertex are left

Maze Generation



- Run dfs from a random starting node & insert neighbours in a random order into the stack.

Topological Sort



- This is for DAG.
- Let vertices represent tasks, a directed edge $A \rightarrow B$ indicates that the parent A must be completed before child B.
- Reversing post-order Dfs gives a valid topo sort.

DFS Postorder: 2 6 9 8 7 5 3 4 1 0

Reverse DFS Postorder: 0 1 4 3 5 7 8 9 6 2

In a post-order DFS, a node is added to the output list **only after** all of its children and downstream descendants have been completely explored and finished.

Because of this rule, children always finish exploring *before* their parents.

